Multi Agent Hospitality System

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Jiyanshi Keshri - EN22CS303026
Manasvi Jain - EN22CS301580
Kratika Tomar - EN22CS301516
Madhuri Prajapati - EN22CS301562
Nikhil Kumar Singh - EN22CS301648
Priyanka Kumari - EN22CS301758

Under the Guidance of

Dr. Hemlata Patel

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Multi Agent Hospitality System

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Jiyanshi Keshri - EN22CS303026
Manasvi Jain - EN22CS301580
Kratika Tomar - EN22CS301516
Madhuri Prajapati - EN22CS301562
Nikhil Kumar Singh - EN22CS301648
Priyanka Kumari - EN22CS301758

Under the Guidance of
Dr. Hemlata Patel

MEDICAPS UNIVERSITY

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Report Approval

The project work **"Multi Agent Hospitality System"** is hereby approved as a creditable study of an engineering application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled "**Multi Agent Hospitality System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology of Computer Applications in 'Computer Science and Engineering' completed under the supervision of **Dr. Hemlata Patel, Asst. Professor, Computer Science & Engineering Department**, Medicaps University, Indore.

Further, We declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Jiyanshi Keshri - EN22CS303026
Manasvi Jain - EN22CS301580
Kratika Tomar - EN22CS301516
Madhuri Prajapati - EN22CS301562
Nikhil Kumar Singh - EN22CS301648
Priyanka Kumari - EN22CS301758

Certificate

I **Dr**. **Hemlata Patel** certify that the project entitled **"Multi Agent Hospitality System"** submitted in fulfillment for the award of the degree of Bachelor of Technology of Computer Applications by **Jiyanshi Keshri, Manasvi Jain, Kratika Tomar, Madhuri Prajapati, Nikhil Kumar Singh, Priyanka Kumari** is the record carried out by her under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr. Hemlata Patel	**Mr. Rishabh Tripathi**
Asst. Professor	Project Leader
Computer Science & Engineering	Datagami Technology Services
Medicaps University, Indore	Private Limited

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express my deepest gratitude to Honorable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya,** Associate Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

We express our heartfelt gratitude to my **External Guide, Mr. Rishabh Tripathi**, Project Lead, Datagami Technology Services Pvt. Ltd as well as to my **Internal Guide, Dr. Hemlata Patel,** Asst. Professor, Department of Computer Science & Engineering, MU, without whose continuous help and support, this project would ever have reached to the completion.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Jiyanshi Keshri - EN22CS303026
Manasvi Jain - EN22CS301580
Kratika Tomar - EN22CS301516
Madhuri Prajapati - EN22CS301562
Nikhil Kumar Singh - EN22CS301648
Priyanka Kumari - EN22CS301758

B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

This project presents a **Multi-Agent Hospitality System** that integrates **Agentic AI, Generative AI, and DevOps practices** to automate and enhance hospitality service management. The system is designed to simulate intelligent collaboration between multiple AI agents responsible for handling various tasks such as customer interaction, booking management, service recommendations, and issue resolution.

The application leverages **Generative AI (LLMs)** to provide dynamic and context-aware responses to user queries, including hotel recommendations, room availability, pricing details, and personalized services. By incorporating **Agentic AI principles**, the system distributes responsibilities across specialized agents (e.g., Booking Agent, Support Agent, Recommendation Agent), enabling structured workflows, decision-making, and efficient task coordination.

The frontend interface is designed to be interactive and user-friendly, allowing seamless communication between users and the AI agents. On the backend, APIs facilitate communication between components and ensure smooth data flow.

From a **DevOps perspective**, the system is containerized using Docker, automated through CI/CD pipelines (e.g., GitHub Actions), and deployed on cloud infrastructure such as AWS EC2. This ensures scalability, reliability, and continuous delivery, making the system production-ready and adaptable to real-world hospitality environments.

Overall, the project demonstrates how the combination of multi-agent systems, generative intelligence, and modern DevOps practices can transform traditional hospitality services into an intelligent, automated, and scalable solution.

Keywords:

Multi-Agent Systems, Agentic AI, Generative AI, Large Language Models (LLMs), Hospitality Management System, Intelligent Automation, AI Agents, Workflow Orchestration, Personalized Recommendations, Conversational AI, DevOps, Docker, CI/CD Pipeline, GitHub Actions, Cloud Deployment, AWS EC2, Scalable Systems, API Integration, Smart Booking System

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	viii
	List of tables	ix
	Abbreviations	x
	Notations & Symbols	xi
Chapter 1	Introduction	
	1.1 Introduction	1
	1.2 Literature Review	2
	1.3 Objectives	2
	1.4 Significance	2
	1.5 Research Design	3
	1.6 Source of Data	3
	1.7 Chapter Scheme	3
Chapter 2	Report on Present Investigation	
	2.1 Experimental Set-up	4
	2.2 Procedures Adopted	4
	2.3 Workflow Integration Summary	5
Chapter 3	System Architecture Design	
	3.1 Workflow Structure	6
	3.2 System Components	7-8
	3.3 Data Flow	7-15
Chapter 4	Implementation	
	4.1 Technologies Used	16
	4.2 Process Flow	17
	4.3 Flow Diagrams	18-19
	4.4 Component Design	20-22
	4.5 Key Design Considerations	23
	4.6 API Catalogues	24
Chapter 5	Data Designs	25-26
	5.1 Data Model	
	5.2 Data Access Mechanism	27-28
	5.3 Data Retention Policies	29
	5.4 Data Migration	29
Chapter 6	DevOps Integration	

	6.1 Docker Containerization	30
	6.2 Github Actions CI/CD Pipeline	31
	6.3 AWS EC2 Cloud Deployment	31
	6.4 Docker Compose	32
	6.5 Key Features	32
	6.6 Security	33
Chapter 7	Results and Discussions	33
Chapter 8	Summary and Conclusions	34
Chapter 9	Future scope	35
	Appendix	35
	Bibliography	35

List of Figures

S.no	Diagram	Fig.no
1.	Architecture Diagram	3.2.1
2.	Activity Diagram	3.2.2
3.	Class Diagram	3.2.3
4.	Component Diagram	3.2.4
5.	Control Flow Diagram	3.2.5
6.	Data Flow Diagram	3.2.6
7.	ER Diagram	3.2.7
8.	State Transition Diagram	3.2.8
9.	Use Case Diagram	3.2.9
10.	Cloud Deployment Diagram	3.2.10

List of Tables

S.no	Tables	Fig.no
1.	System Component Table	3.2
2.	API Catalogue	4.6

Abbreviations

AI	Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
CI/CD	Continuous Integration and Deployment
AWS	Amazon Web Services
EC2	Elastic Compute Cloud
JSON	JavaScript Object Notation
UI	User Interface

Notations & Symbols

Symbol / Notation	Meaning
→	Process/Data flow (used in workflow descriptions)
{}	JSON object (structured response from LLM)
[]	Array/List structure in JSON
:	Key-value separator in JSON
/endpoint	API route notation (e.g., /add, /patients)
GET	HTTP method for retrieving data
POST	HTTP method for sending data
st.*	Streamlit function notation (e.g., st.chat_message)
Dockerfile	Container configuration file
YAML	CI/CD pipeline configuration format
EC2	Cloud instance notation in deployment

Chapter-1

Introduction

1.1 Introduction

With rapid advancements in **Artificial Intelligence (AI)**, the hospitality industry is undergoing a significant transformation toward automation and intelligent service delivery. Traditional hospitality systems often rely on manual processes or basic rule-based software, which limits efficiency, personalization, and scalability.

This project introduces a **Multi-Agent Hospitality System** that leverages **Generative AI and Agentic AI** to automate and optimize hospitality services such as booking management, customer interaction, and personalized recommendations. Unlike conventional systems, this solution utilizes multiple intelligent agents working collaboratively, each responsible for specific tasks like handling reservations, responding to customer queries, and suggesting services.

The system integrates **Large Language Models (LLMs)** to generate dynamic, context-aware responses, ensuring a more interactive and user-friendly experience. Additionally, **DevOps practices** such as containerization, CI/CD pipelines, and cloud deployment are incorporated to ensure scalability, reliability, and real-world applicability.

By combining multi-agent collaboration, generative intelligence, and modern deployment strategies, the project presents a smart and efficient solution for next-generation hospitality management.

1.2 Literature Review

Recent studies highlight the growing role of AI in transforming the hospitality and service industries:

- **AI in Hospitality**: Research shows AI improves customer experience through automation, personalization, and predictive analytics in hotel and service management systems.
- **Virtual Assistants in Services**: AI-powered assistants are widely used for handling customer queries, bookings, and support, significantly reducing manual workload and improving response time.
- **Chatbots and Limitations**: Traditional chatbots are often rule-based and lack contextual understanding, resulting in generic and sometimes irrelevant responses.
- **Generative AI & LLMs**: Models like GPT and LLaMA enable real-time, intelligent, and context-aware communication, allowing systems to generate personalized recommendations and structured outputs.
- **Agentic AI Systems**: Multi-agent architectures distribute tasks among specialized agents, improving system efficiency, modularity, and decision-making capabilities.
- **DevOps in AI Systems**: Modern deployment techniques such as Docker, CI/CD pipelines, and cloud platforms ensure scalability, maintainability, and continuous integration of AI applications.

Limitations of Existing Systems

- Lack of personalization in traditional systems
- Static and rule-based responses
- Poor scalability and integration capabilities
- Limited automation and workflow management

1.3 Objectives

1. To develop a **Multi-Agent system** for hospitality management.
2. To integrate **Generative AI (LLMs)** for dynamic and intelligent responses.
3. To design an **agent-based workflow** for task distribution and coordination.
4. To build an interactive and user-friendly frontend interface.
5. To implement **Docker-based containerization** for portability.
6. To automate workflows using **CI/CD pipelines (GitHub Actions)**.
7. To deploy the system on **cloud platforms (AWS EC2)** for scalability and accessibility.

1.4 Significance

This project provides a modern solution to challenges in hospitality management by introducing intelligent automation and personalized service delivery. It enhances customer experience through quick and accurate responses while reducing operational overhead for service providers.

The use of multi-agent architecture ensures efficient task handling and scalability, making the system adaptable to various hospitality environments such as hotels, resorts, and service platforms. Furthermore, the integration of DevOps practices ensures that the system is reliable, maintainable, and ready for real-world deployment.

1.5 Research Design

- **Architecture**:
  A modular architecture consisting of a frontend interface, backend services, multiple AI agents, LLM integration, and a DevOps deployment layer.

- **Workflow**:
  User queries are received through the interface → routed to appropriate agents → processed using structured prompts → LLM generates responses → results are delivered to the user in a structured format.
- **Methodology**:
  Agent-based design with role-specific responsibilities such as booking, recommendation, and support.
- **Evaluation Criteria**:
  - Response accuracy and relevance
  - System scalability
  - Workflow efficiency
  - Deployment reliability

1.6 Source of Data

- **Primary Data**:
  User inputs such as booking requests, service inquiries, and feedback.
- **Secondary Data**:
  - Research papers and journals on AI and hospitality
  - Documentation of LLM APIs
  - Online datasets related to hotel services and customer interactions

1.7 Chapter Scheme

The report is organized into the following chapters:

- **Chapter 1**: Introduction, Literature Review, Objectives, and Research Design
- **Chapter 2**: System Analysis and Requirements Specification
- **Chapter 3**: System Design (Architecture, UML Diagrams, Data Models)
- **Chapter 4**: Implementation (Frontend, Backend, AI Agents, DevOps Setup)
- **Chapter 5**: Testing and Results
- **Chapter 6**: Conclusion and Future Scope

Chapter – 2

Report on Present Investigation

2.1 Experimental Set-up

The Multi-Agent Hospitality System is developed using a modular architecture that combines user interface, backend processing, AI agents, and cloud deployment. The system is designed to ensure smooth interaction between users and intelligent agents.

The frontend provides an interactive interface where users can make bookings, ask queries, and receive recommendations. The backend, built using Python, handles request processing, agent coordination, and communication with AI models.

The intelligence of the system is powered by Generative AI (LLMs), which generate dynamic and context-aware responses. A multi-agent approach is used to divide responsibilities among specialized agents.

Key Components:

- Frontend: Interactive UI (Streamlit/Web Interface)
- Backend: Python-based logic and API handling
- AI Models: LLM APIs for intelligent response generation
- Agents: Booking, Recommendation, and Support agents
- DevOps: Docker, GitHub Actions (CI/CD), AWS EC2 deployment

This setup ensures scalability, flexibility, and real-world usability.

2.2 Procedures Adopted

The system is developed and deployed using a structured process that integrates AI capabilities with DevOps practices.

Initially, the application is containerized using Docker to maintain consistency across different environments. A lightweight Python image is used, and all dependencies are installed within the container.

To automate development and deployment, a CI/CD pipeline is implemented using GitHub Actions. This ensures that every code update is automatically tested, built, and deployed without manual intervention.

Deployment is carried out on AWS EC2, where the application runs inside a Docker container and is accessible through a public IP.

Important Steps:

- Containerization using Docker for portability
- Automated testing and deployment using CI/CD pipeline
- Cloud deployment on AWS EC2
- Secure handling of API keys using environment variables

Security and scalability are maintained by avoiding hardcoding sensitive data and enabling easy horizontal scaling through containers.

2.3 Workflow Integration Summary

The system follows a seamless workflow where all components work together to deliver intelligent hospitality services.

When a user submits a request, it is processed by the backend and routed to the appropriate agent. The agent prepares a structured input for the AI model, which then generates a response. The result is formatted and displayed to the user through the frontend.

Workflow Highlights:

- User input received through frontend
- Request routed to the relevant AI agent
- LLM generates context-aware response
- Output processed and displayed to user
- DevOps pipeline ensures continuous updates and deployment

Chapter 3

System Architecture Design

3.1 Workflow Structure

The Multi-Agent Hospitality System follows a structured workflow combined with intelligent agent-based decision-making. The goal is to ensure that user requests are processed efficiently and responses are accurate, personalized, and well-organized.

The system operates through a step-by-step pipeline where user input is analyzed, processed, and converted into meaningful output using AI models. At the same time, multiple agents collaborate to handle different parts of the task.

Key Workflow Concepts:

- **Stepwise Processing:** Input → Processing → AI Response → Structured Output
- **Agent-Based Execution:** Specialized agents handle different responsibilities (booking, support, recommendations)
- **Task Decomposition:** Complex queries are broken into smaller tasks for better accuracy
- **Routing Mechanism:** Requests are directed to the most relevant agent
- **Hybrid Approach:** Combines fixed workflows with dynamic agent decision-making
- **State Management:** Maintains session data to track user interactions

This workflow ensures both **automation and flexibility**, making the system efficient and intelligent.

3.2 System Components

The system is divided into multiple components, each responsible for a specific function. Together, they form a complete and scalable architecture.

Component	Description
Frontend	Interactive user interface (Streamlit/Web UI) for user interaction, chat, and displaying results
Backend	Python-based logic for handling requests, managing agents, and processing responses
AI Models	Generative AI (LLMs) used to generate intelligent, context-aware responses
Agents	Specialized units like Booking Agent, Recommendation Agent, and Support Agent
DevOps Layer	Docker for containerization and GitHub Actions for CI/CD automation
Cloud	AWS EC2 for hosting, deployment, and scalability

(Table.1)

Important Points:

- Each component is loosely coupled → easy to update and scale
- Agents improve modularity and system intelligence
- Cloud deployment ensures real-world usability

3.3 Data Flow

The data flow in the system represents how information moves between different components from input to output.

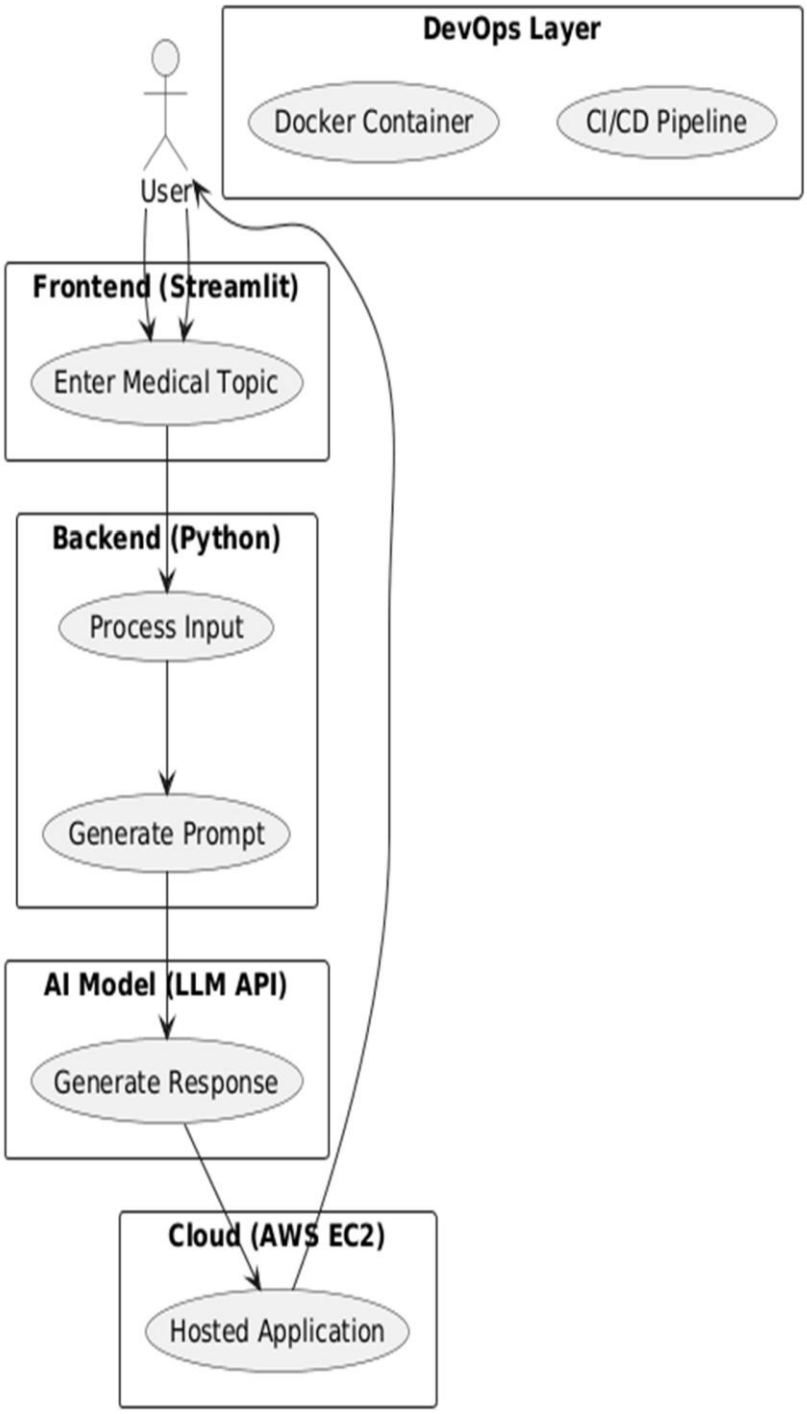
When a user submits a request, it is first captured by the frontend and sent to the backend. The backend processes the request and identifies which agent should handle it. The selected agent then prepares a structured prompt and sends it to the AI model.

The AI model generates a response, which is processed and formatted before being sent back to the frontend for display.

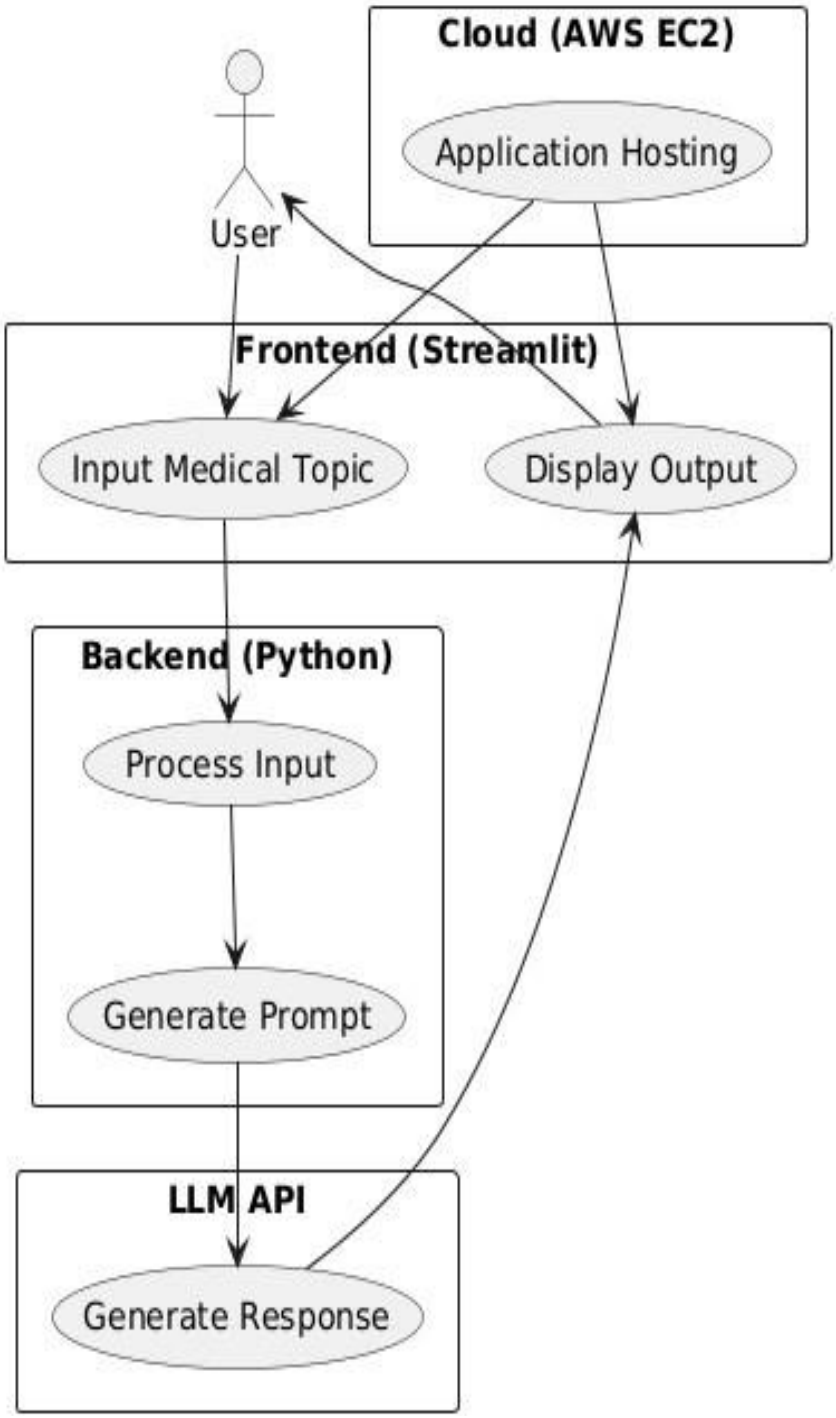
Data Flow Steps:

1. User enters request through frontend
2. Backend receives and processes input
3. Request routed to appropriate agent
4. Agent communicates with AI model (LLM)

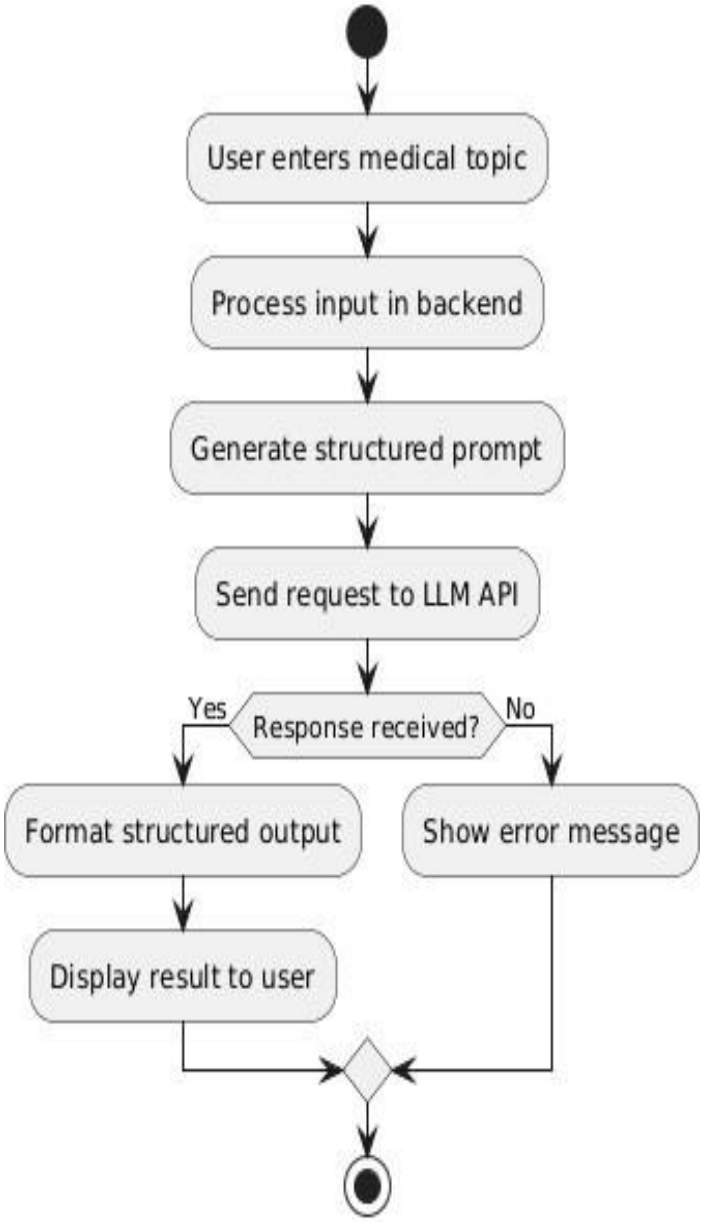
3.2.1 Architecture Diagram Description



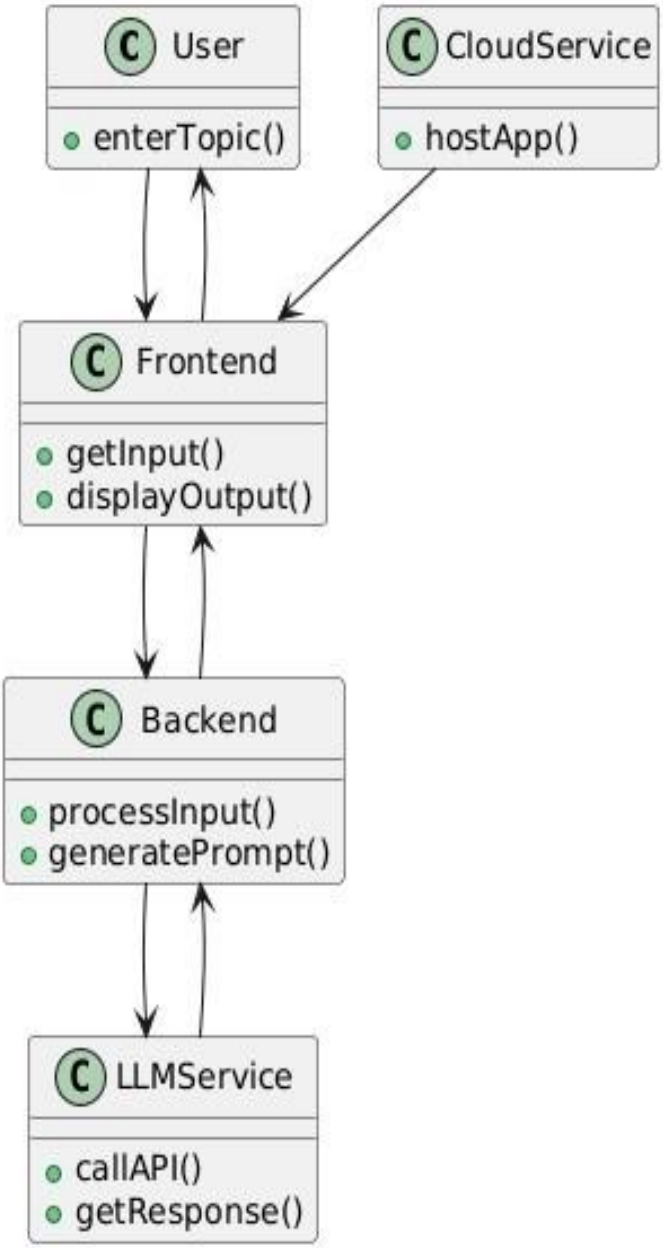
3.2.2 Data Flow Diagram



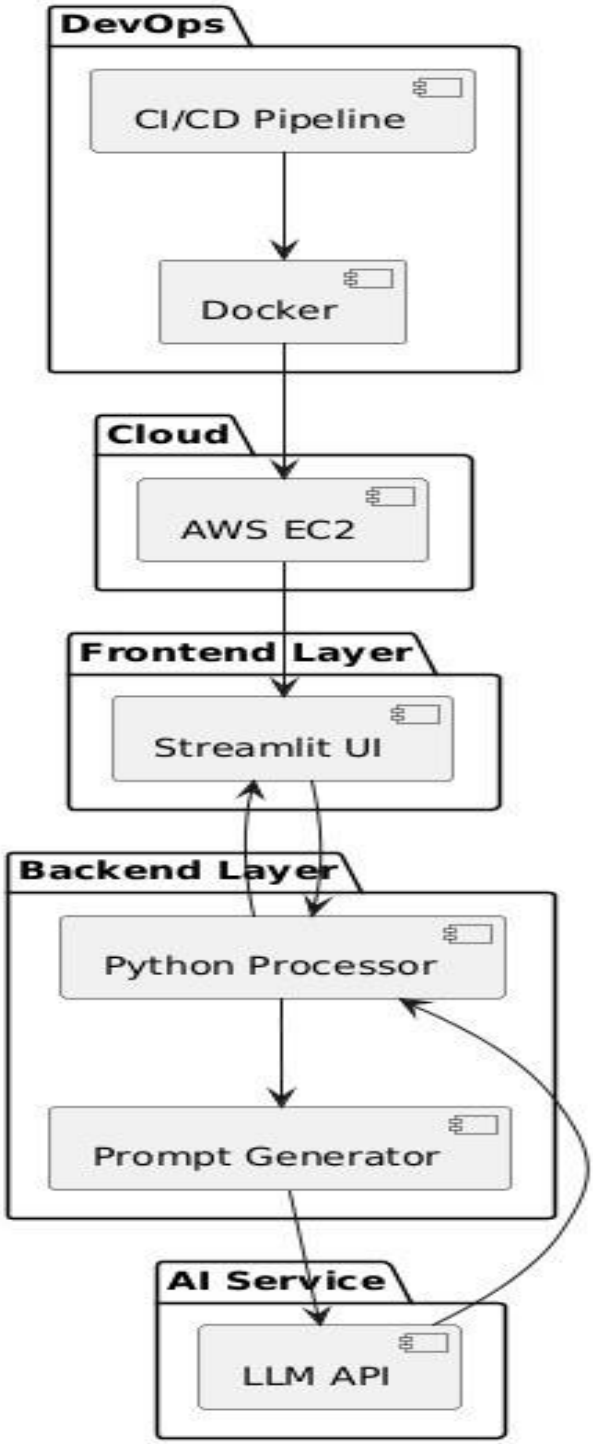
3.2.2 Activity Diagram



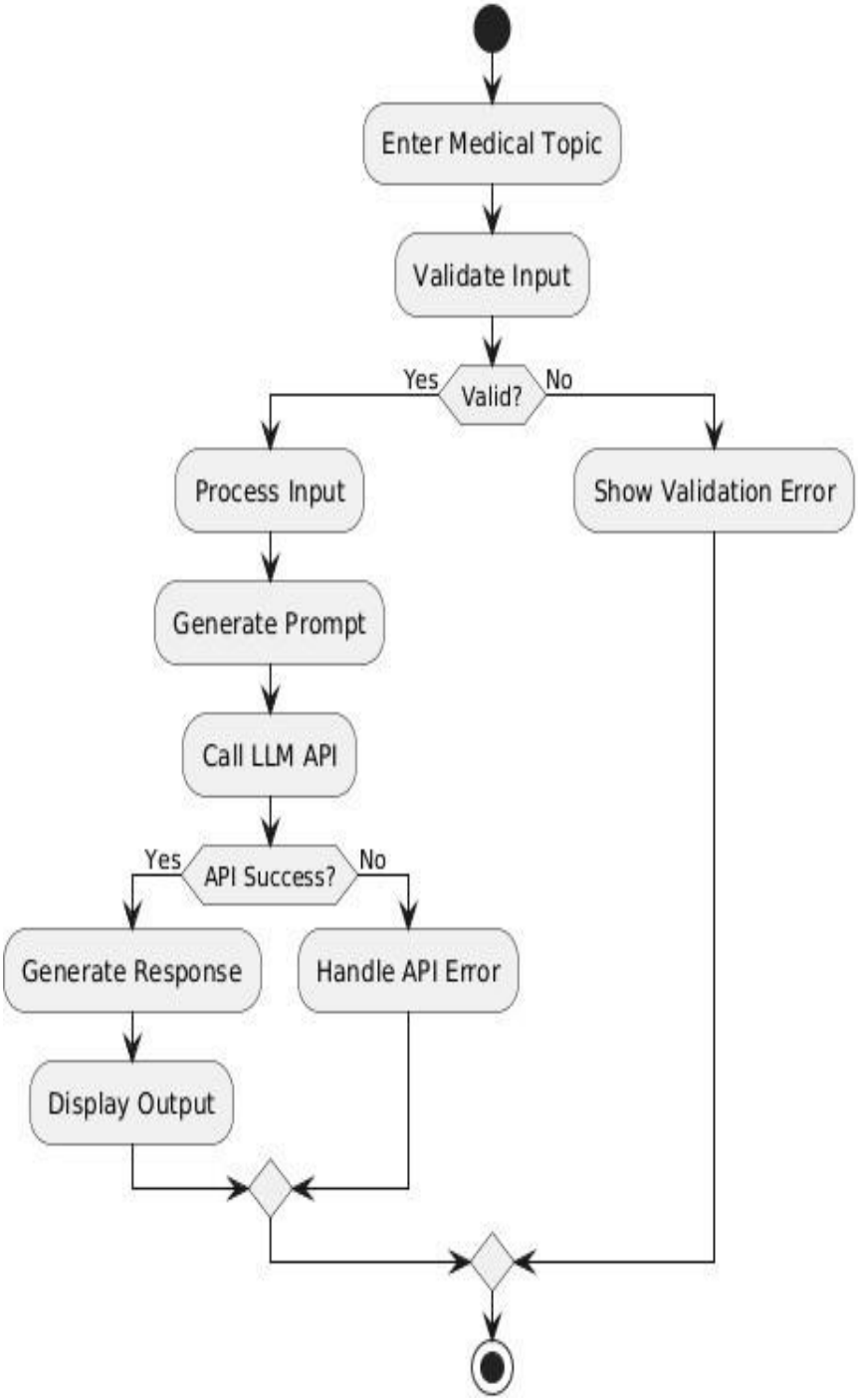
3.2.3 Class Diagram



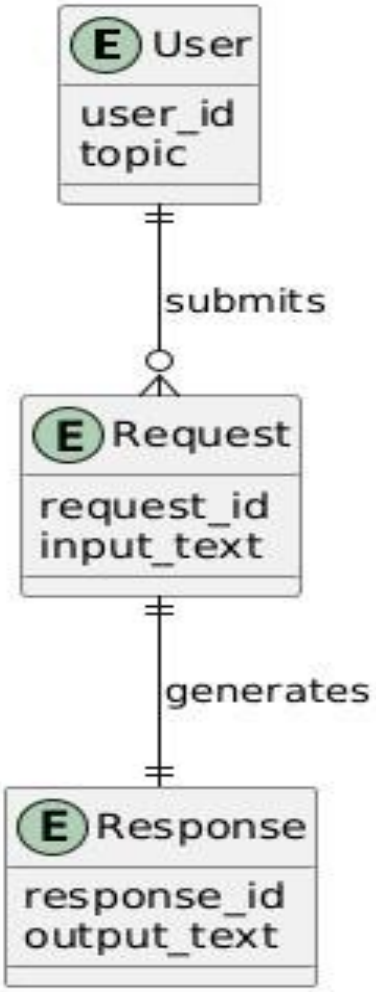
3.2.5 Component Diagram



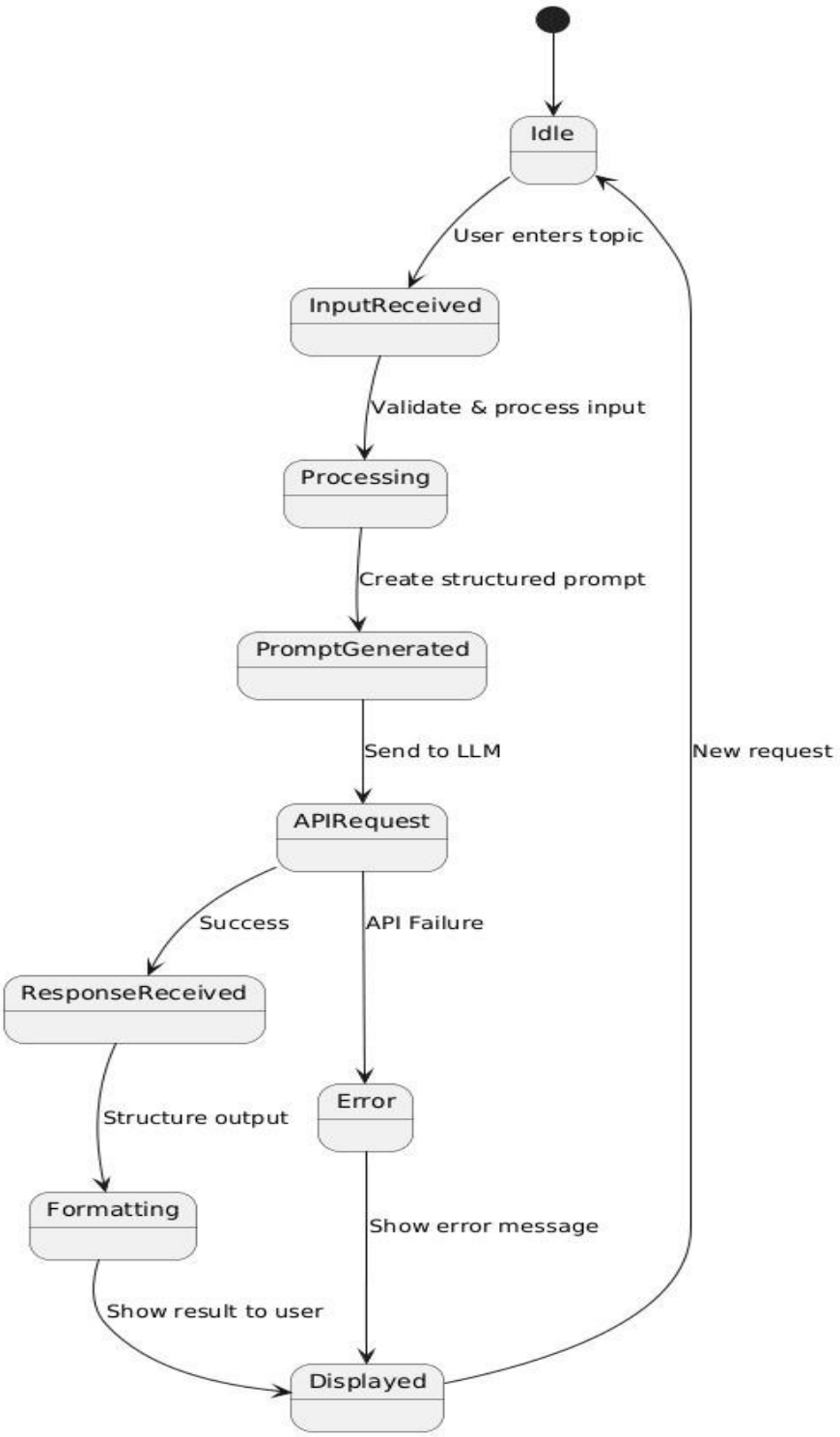
3.2.6 Control Flow Diagram



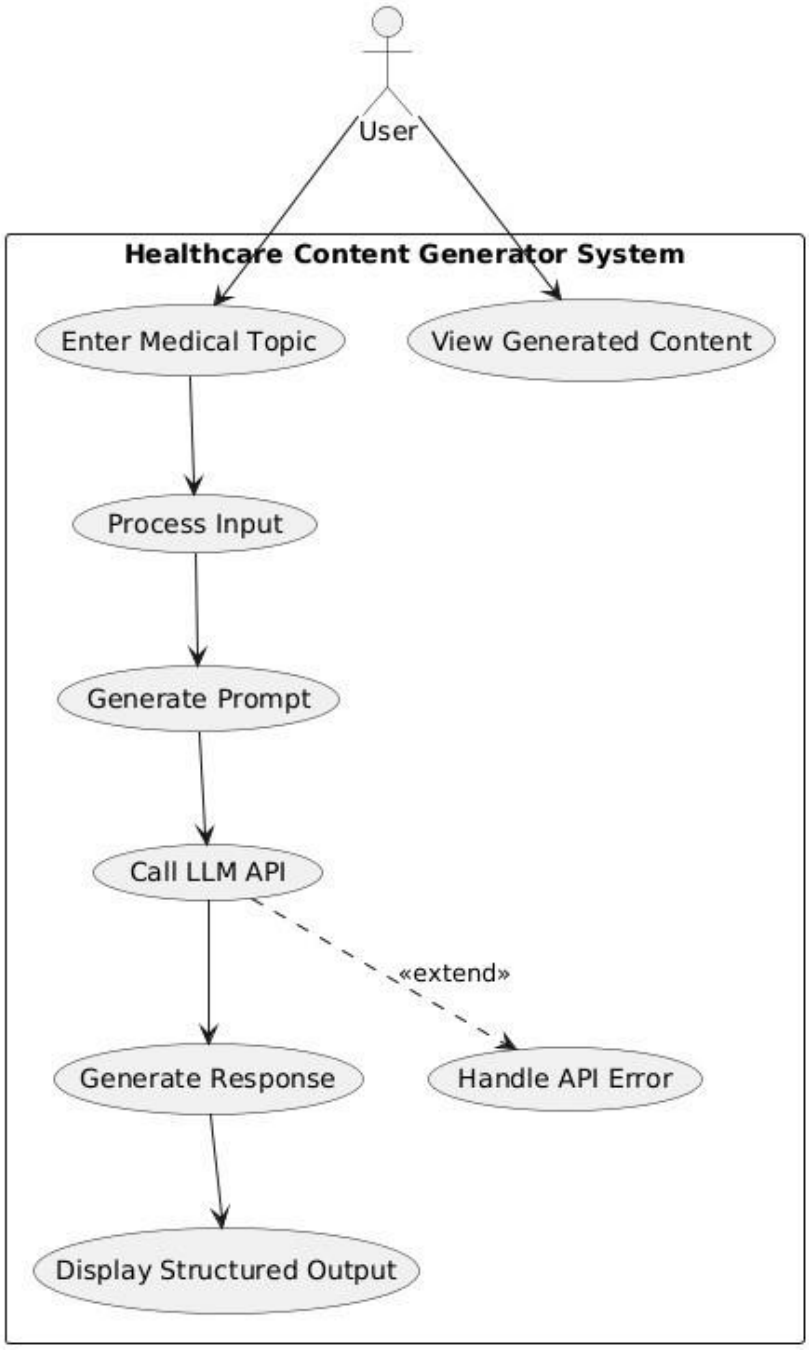
3.2.7 ER Diagram



3.2.8 State Transition Diagram

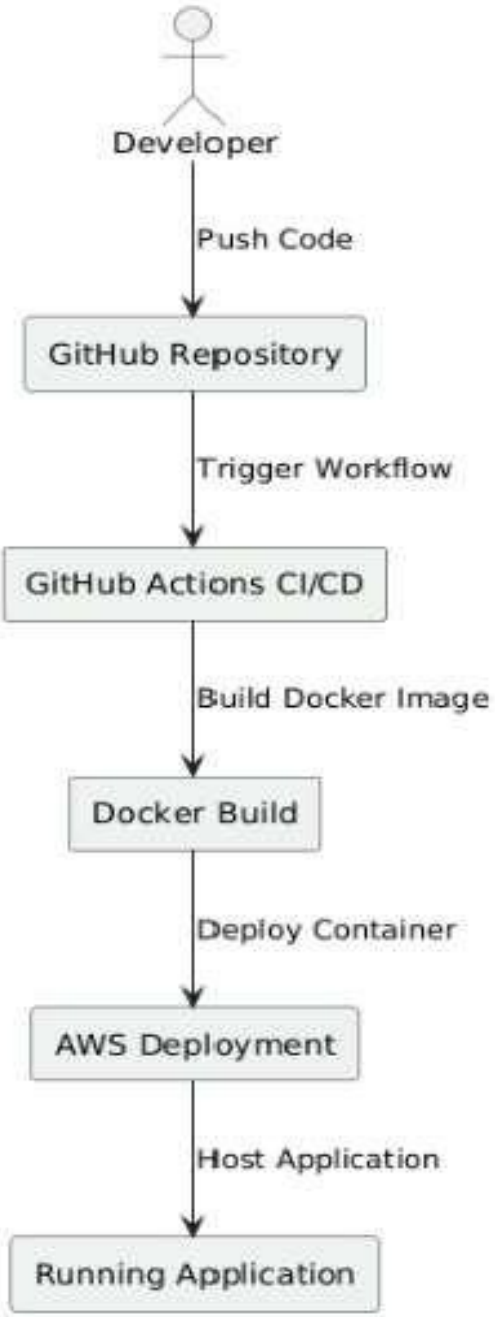


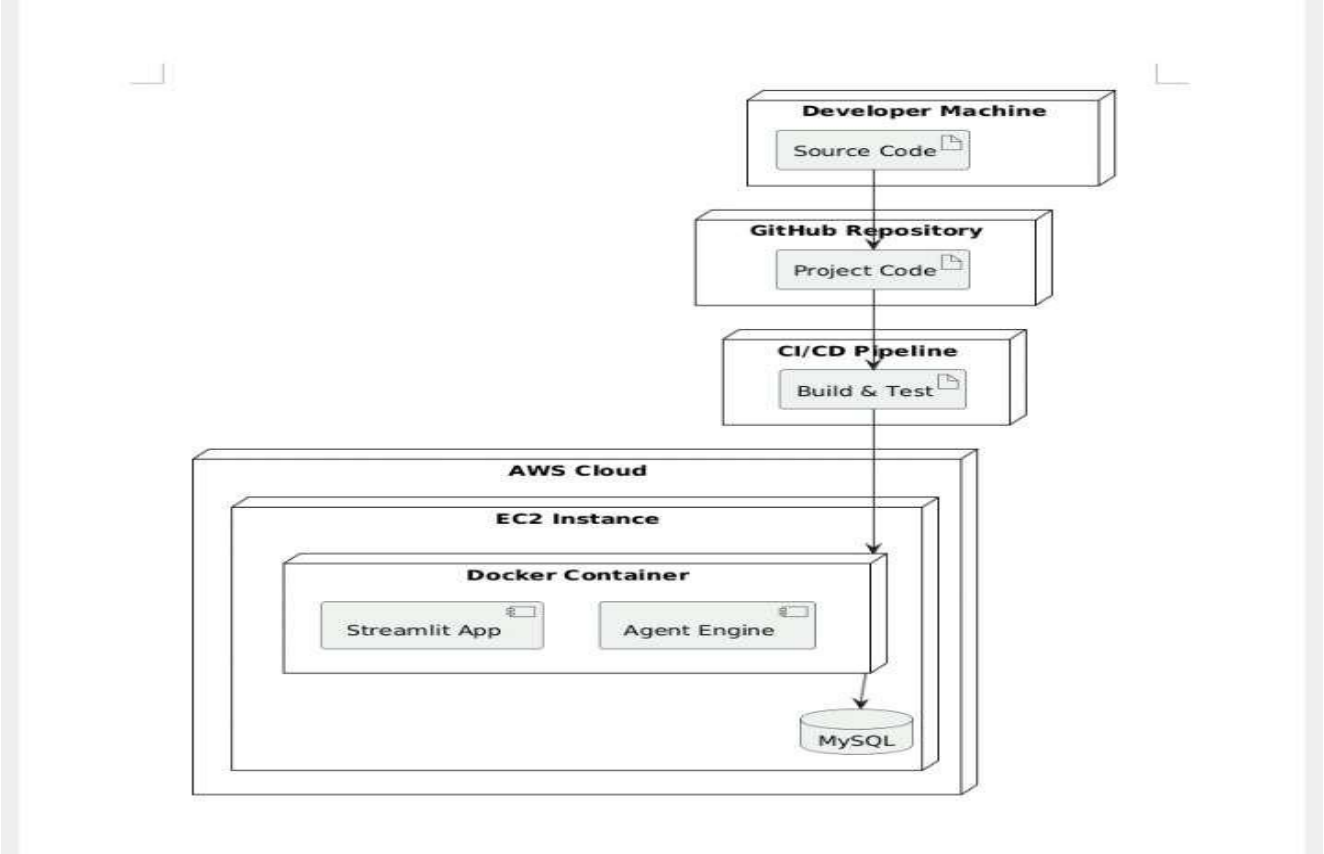
3.2.9 Use Case Diagram



3.2.10 CI/CD and Cloud Deployment Design

The system includes DevOps automation through CI/CD pipelines and cloud deployment support. Source code changes can be integrated with GitHub Actions workflows to automate build, test, and deployment stages. The containerized application can further be deployed on AWS infrastructure for scalable hosting and production readiness.


Developer
Push Code
GitHub Repository
Trigger Workflow
GitHub Actions CI/CD
Build Docker Image
Docker Build
Deploy Container
AWS Deployment
Host Application
Running Application



Data Flow:

- User enters a medical query in the Streamlit frontend.
- Backend processes input, generates a structured prompt.
- Prompt sent to Groq LLaMA API.
- LLM returns structured response (JSON: causes, symptoms, precautions, treatment).
- Backend parses and organizes response.
- Frontend displays categorized output to user.

Chapter – 4

4.1 Technologies Used

Backend: Python 3.9+, Groq.

Frontend: Streamlit.

AI Model: Groq LLaMA-based models.

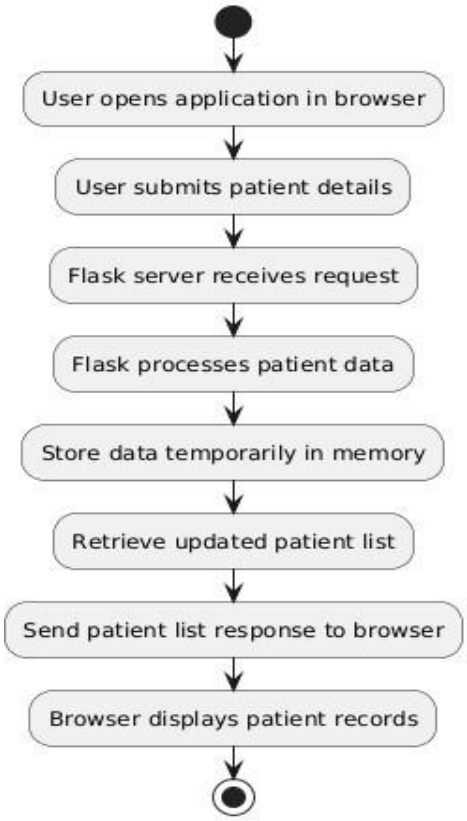
DevOps: Docker, GitHub Actions, AWS EC2

4.2 Process Flow

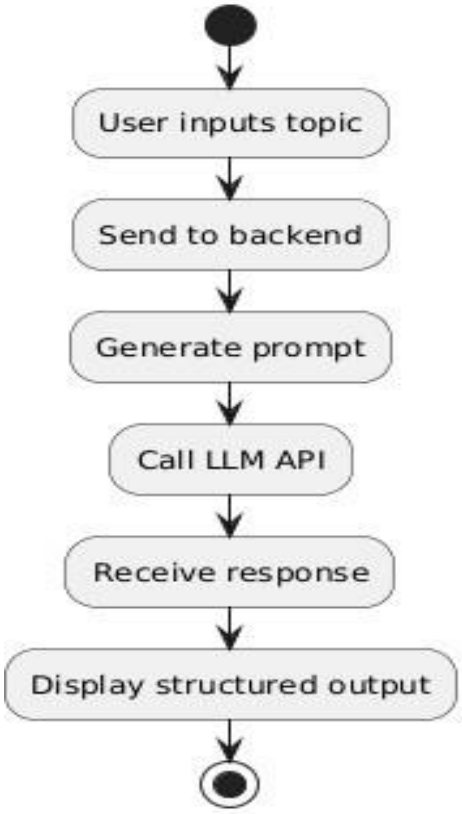
1. User opens the application in a browser.
2. User submits patient details via a form.
3. Flask server processes the input and stores it temporarily.
4. Patient list is updated and displayed to the user.

4.3 Flow Diagrams

4.3.1 Information flow



4.3.2 Process Flow Diagram



4.4 Components Design

- **Flask Application (app.py):** Core backend logic.
- **HTML Templates:** User interface.
- **Docker Container:** Encapsulates the application environment.
- **GitHub Repository:** Source code management.
- **CI/CD Pipeline:** Automates build and deployment.
- **AWS EC2 Instance:** Hosts the containerized application.

4.5 Key Design Considerations

- Simplicity and ease of deployment.
- Container-based architecture for environment consistency.
- Scalability readiness with Kubernetes.
- Automation through CI/CD pipelines.

- Security considerations including AWS security groups and controlled access.

4.6 API Catalogue

Endpoint	Method	Description
/	GET	Home page
/add	GET/POST	Add patient details
/patients	GET	View patient records

Chapter – 5

Data Design

5.1 Data Model

The system uses a simple and flexible data model to handle user interactions, bookings, and agent responses. Since the application is primarily AI-driven, the focus is on handling request-response data efficiently rather than complex database relationships.

Main Entities:

- **User:**
  - Name (String)
  - Email (String)
  - Contact (String)
- **Booking:**
  - Booking ID (String)
  - Room Type (String)
  - Check-in Date (Date)
  - Check-out Date (Date)
  - Status (String)
- **Request/Query:**
  - Query ID (String)
  - User Input (String)
  - Query Type (String: Booking / Recommendation / Support)
- **Response:**
  - Response ID (String)
  - Generated Output (Text)
  - Timestamp (DateTime)

This model supports smooth handling of user requests and AI-generated outputs.

5.2 Data Access Mechanism

Currently, the system uses in-memory data handling for simplicity and faster processing.

Data is stored temporarily using Python data structures (lists, dictionaries, session state)

- Backend modules handle data processing and routing between agents
- APIs are used to send and receive data between frontend, backend, and AI models

This approach ensures **quick access and real-time interaction** without database overhead.

5.3 Data Retention Policies

The system follows a temporary data storage approach in its current implementation.

- Data is stored only during the active session
- No permanent storage is maintained
- All data is cleared when the application is restarted

This approach improves simplicity and avoids unnecessary storage complexity but is not suitable for long-term data tracking.

5.4 Data Migration

At present, data migration is not applicable due to the absence of persistent storage.

However, in future enhancements:

- Integration with databases like MySQL or MongoDB can be implemented
- Data such as bookings, user profiles, and history can be stored permanently
- Migration strategies can be applied to move in-memory data to database systems

This will improve **data reliability, tracking, and scalability** for real-world deployment.

Chapter 6

DevOps Integration

6.1 Docker Containerization

Docker is used to package the application into containers, ensuring a consistent and portable environment across systems.

- Easy deployment
- No dependency issues

6.2 GitHub Actions CI/CD Pipeline

Automates build, testing, and deployment whenever code is updated.

- Reduces manual work
- Ensures continuous integration

6.3 AWS EC2 Cloud Deployment

Application is hosted on AWS EC2, making it accessible over the internet.

- Scalable and reliable
- Real-world deployment

6.4 Docker Compose

Used to manage multiple services (frontend, backend) in one setup.

- Simplifies execution
- Easy configuration

6.5 Key Features

- Automated deployment
- Scalable architecture
- Secure data handling
- Fast and efficient workflow

Chapter 7

Results and Discussions

The Multi-Agent Hospitality System was successfully implemented and tested for handling user queries, bookings, and recommendations. The system demonstrated the ability to generate **accurate, context-aware, and structured responses** using Generative AI.

The multi-agent approach improved task handling by distributing responsibilities among specialized agents, resulting in better efficiency and faster response time. The integration of DevOps ensured smooth deployment and reliable system performance.

Key Observations:

- Intelligent and dynamic responses using LLMs
- Efficient task handling through multiple agents
- Smooth deployment using Docker and CI/CD
- Good user interaction through simple UI

Chapter - 8

Summary and Conclusions

This project successfully combines **Agentic AI, Generative AI, and DevOps practices** to create an intelligent hospitality system. The system automates key services such as booking management, query handling, and recommendations.

The use of multi-agent architecture enhances modularity and scalability, while cloud deployment ensures real-world applicability. Overall, the system proves to be an efficient, scalable, and modern solution for hospitality management.

Chapter - 9

Future Scope

The project can be further enhanced with additional features and improvements:

- Integration with databases for persistent data storage
- Advanced personalization using user history and preferences
- Voice-based interaction and chatbot integration
- Mobile application development
- Integration with real hotel APIs for live data
- Enhanced security and authentication mechanisms

Appendix

The appendix includes supporting materials used in the project:

- Sample code snippets (Dockerfile, CI/CD pipeline)
- API request and response formats
- Screenshots of the application interface
- Test cases and outputs

Bibliography

- Research papers on AI in hospitality and automation
- Documentation of Generative AI models (LLMs)
- Docker official documentation
- GitHub Actions documentation
- AWS EC2 documentation
- Online tutorials and technical blogs related to AI and DevOps